

Lecture 8 — Execution of Instruction

1. What Is an Instruction?

An instruction is a command that tells the CPU to perform a specific operation.

For example:

ADD A, B

This instruction tells the CPU to add the values stored in A and B.

Other examples include:

Instruction	Meaning
ADD	Add values
SUB	Subtract values
LOAD	Load data
STORE	Store data
JUMP	Change the sequence of execution
COMPARE	Compare values

These are illustrative assembly-style instructions. Actual instruction formats depend on the processor's instruction set architecture.

A simple example

Suppose we have the following instruction:

ADD 10, 20

The CPU needs to:

1. Fetch the instruction.
2. Decode the instruction.
3. Obtain the required operands.

4. Perform the addition.
5. Store or otherwise use the result.

The result is:

$$10 + 20 = 30$$

But how does the CPU perform these steps internally?

That's what we are going to learn.

2. The Instruction Cycle

The CPU repeatedly executes instructions using a process commonly called the instruction cycle.

Its basic stages are:

STAGE 1

Fetch

Retrieve the next instruction from memory.

STAGE 2

Decode

Determine what the instruction means.

STAGE 3

Execute

Perform the required operation.

STAGE 4

Store / Write back

Save the result when required.

The cycle repeats for subsequent instructions.

Some textbooks describe the basic cycle using only three stages: fetch, decode and execute. Others show additional stages, such as memory access and write-back, particularly when explaining modern processors.

For our introductory example, we will focus on the classic fetch-decode-execute process.

3. Stage One — Fetch

Fetch means retrieving an instruction from memory.

The CPU needs to know which instruction to execute next.

This is where the Program Counter (PC) comes into action.

Suppose memory contains:

Memory address	Instruction
1000	LOAD A
1001	LOAD B
1002	ADD A, B
1003	STORE C

Assume the PC contains:

PC = 1000

This means the next instruction is located at memory address 1000.

Step-by-step fetching

Step 1: Copy the address

The address stored in the PC is copied into the MAR.

PC = 1000

↓

MAR = 1000

Step 2: Access memory

The CPU sends a memory read request for address 1000.

Memory retrieves the instruction stored at that address.

Step 3: Transfer the instruction

The instruction travels from memory through the MDR.

Memory



MDR

Step 4: Store the current instruction

The instruction is transferred from the MDR into the CIR.

MDR



CIR

Now the CPU has the instruction it needs to process.

Step 5: Update the PC

The PC is updated to point to the next instruction in the normal sequential execution sequence.

For our simplified example:

PC = 1001

The PC may instead be changed to a different address when a jump, branch or other control-flow instruction is executed.

Fetch stage — summary

PC → MAR → Memory → MDR → CIR

The PC is also updated to identify the next instruction.

4. Stage Two — Decode

Decode means interpreting the fetched instruction to determine what operation must be performed.

The Control Unit (CU) plays a central role in this stage.

Let's say the CIR contains:

ADD A, B

The CPU needs to understand two things:

- Opcode: What operation should be performed?
- Operands: Which data values are involved?

For this instruction:

Part	Meaning
------	---------

ADD Opcode (addition operation)

A, B Operands (values to be added)

The Control Unit decodes the instruction and coordinates the necessary operations.

For example, it determines that:

- The CPU needs the values of A and B.
- The ALU must perform addition.
- The result needs to be stored or passed to a subsequent operation.

Decode stage

CIR

ADD A, B

Control Unit

Interprets the instruction

Required operation: ADD

Operands: A and B

An important distinction:

The Control Unit decodes and coordinates. The ALU performs the arithmetic operation.

5. Stage Three — Execute

Execute means carrying out the operation specified by the instruction.

Let's continue our example.

Suppose:

A = 10

B = 20

The decoded instruction is:

ADD A, B

The CPU obtains the required values and sends them to the ALU.

The ALU performs:

$$10 + 20 = 30$$

The result may be stored in a register or passed to another part of the CPU.

For example, in a simplified accumulator-based CPU:

$$AC = 30$$

How the ALU executes addition

OPERAND 1

10

OPERAND 2

20

ALU

$$10 + 20$$

30

RESULT REGISTER

30

Notice that the ALU doesn't decide what calculation to perform by itself. The decoded instruction and control signals determine the required operation.

6. Stage Four — Store or Write Back

After execution, the result may need to be stored.

Consider this instruction:

ADD A, B

If the result is 30, it might be written into a destination register.

For example:

C = 30

In a simplified computer model, the result could be held in the accumulator and then stored in memory using a separate **STORE** instruction.

For example:

ADD A, B
STORE C

The first instruction calculates the result. The second instruction stores it in the memory location associated with C.

This is why the execution of an instruction and storing a result are related but distinct operations.

7. Complete Example — Executing Instructions Step by Step

Now let's use everything we've learned.

Suppose we have the following program:

LOAD A
LOAD B
ADD A, B
STORE C

Assume:

A = 15
B = 25

And our simplified memory looks like this:

Address	Instruction
1000	LOAD A
1001	LOAD B
1002	ADD A, B
1003	STORE C

We will follow the CPU as it executes these instructions.

Interactive instruction execution

Select each step to see what the CPU is doing.

Instruction 1: LOAD A

CPU register snapshot

PC 1000

MAR 1000

MDR LOAD A

CIR LOAD A

AC 15

The CPU fetches LOAD A, decodes it, retrieves the value of A from memory and loads it into the accumulator.

[Previous](#)

Step 1 of 4

[Next](#)

This is a teaching simulation, not a literal hardware trace. The register values are illustrative snapshots; real CPUs have different instruction formats and execution stages.

The important thing to notice is that the CPU does not execute the entire program as one giant operation. It executes a sequence of instructions, each of which goes through the necessary stages.

8. What Happens When an Instruction Is a Conditional Jump?

Not every instruction performs arithmetic.

Consider:

IF A == B, JUMP 2000

This is an illustrative conditional instruction.

The CPU must:

1. Fetch the instruction.
2. Decode the condition and target.
3. Compare the required values.
4. Determine whether the condition is true.
5. Change the PC if the jump condition is satisfied.

For example:

A = 10

B = 10

Since the values are equal, the CPU takes the jump and updates the PC to address 2000.

If they were different, execution would continue along the normal sequence.

This shows why the PC is important: it controls which instruction the CPU fetches next.

9. The Role of Each Component During Execution

Component	Role
PC	Identifies the next instruction address
MAR	Holds the memory address being accessed
MDR	Temporarily holds data or instructions transferred to/from memory
CIR	Holds the current instruction in a simplified model
Control Unit	Decodes instructions and coordinates operations
ALU	Performs arithmetic and logical operations
Registers	Hold immediate values, addresses and intermediate results
Memory	Stores instructions and data
Clock	Provides timing signals that coordinate synchronous operations

One important detail: modern CPUs often use pipelines, multiple execution units, caches and other optimizations. Therefore, the simple cycle is a conceptual model for understanding instruction execution, not a complete description of every modern processor.

10. Common Exam Questions

Q1. What is an instruction cycle?

The instruction cycle is the sequence of operations through which the CPU fetches, decodes and executes program instructions.

Q2. What are the three basic stages of instruction execution?

- Fetch
- Decode
- Execute

Q3. What is the role of the Program Counter?

The PC holds the address of the next instruction to be fetched.

Q4. What is the role of the Control Unit?

The Control Unit interprets instructions and coordinates the activities required for their execution.

Q5. What is the difference between the ALU and the Control Unit?

The ALU performs arithmetic and logical operations, while the Control Unit directs and coordinates instruction execution.

Q6. What is the purpose of the MDR?

The MDR temporarily holds data or instructions being transferred between the CPU and memory.

Final revision

FETCH

Retrieve the instruction

DECODE

Interpret the instruction

EXECUTE

Perform the operation

WRITE BACK

Save the result if needed

Remember: Fetch-decode-execute is the central idea. The registers and control signals help the CPU perform these stages.